USER GUIDE & REFERENCE MANUAL

oxide-sloc the complete operator's manual

A Rust-based local code-analysis tool — IEEE 1045-1992 SLOC counting, test-function detection, COCOMO effort modelling, git hotspots, multi-format coverage import, and HTML / PDF / CSV / Excel reporting, from CLI to LAN server.

APPLIES TO	EDITION	LICENSE	AUDIENCE
v1.5.73 & later	June 2026	AGPL-3.0-or-later	Operators & DevOps

CONTENTS

Every entry is a link — click to jump. Use your browser's Find (Ctrl/Cmd + F) to search the whole manual.

01 **Overview & concepts**
What it is, IEEE 1045, comparisons

02 **Installation**
Package managers, Docker, source, air-gapped

03 **Quick start**
First scan in under a minute

04 **CLI reference**
Every command & flag

05 **Web UI & the scan flow**
The guided 4-step scan

06 **Configuration reference**
Every .oxide-sloc.toml key

07 **Metrics glossary**
SLOC, COCOMO, complexity, hotspots

08 **Output formats**
HTML, PDF, CSV, Excel, JSON

09 **Metrics REST API**
Endpoints, badges, OpenAPI

10 **Local vs server mode**
Behavioural differences

11 **LAN server deployment**
Docker, systemd, TLS, auth

12 **CI/CD integration**
Actions, Jenkins, GitLab, artifacts

13 **AI / MCP integration**
Model Context Protocol tools

14 **Git integration**
Webhooks, browser, submodules

15 **Maintenance & operations**
Updating, vendoring, logs

16 **Troubleshooting**
Common failures & fixes

17 **Acronyms & glossary**
Every term, expanded

A **Supported languages (60)**

B **Environment variable reference**

C **Repository & output layout**

SECTION 01

Overview & concepts

oxide-sloc is a single self-contained Rust binary that measures the size, structure, test coverage and historical churn of a codebase, then renders the result as a report you can read, archive, or publish. It runs three ways from one executable: a command-line tool, a local or LAN web application, and a Model Context Protocol server callable by AI agents.

What it does

At its core oxide-sloc answers *"how big, and how healthy, is this code?"* — far more precisely than a raw line count. A single run produces:

- **Physical SLOC** per the IEEE 1045-1992 standard, classifying every line as code, comment, or blank under configurable policies.
- **Symbol counts** — functions, classes, variables, and imports — detected lexically per language.
- **Effort & cost** via a COCOMO I estimate, plus an approximate cyclomatic-complexity figure.
- **Duplication signals** — ULOC / DRYness and duplicate-file detection.
- **Git Hotspots** — files ranked by `code lines × commit activity` to surface refactor candidates.
- **Test metrics** — lexical test-function detection across 60 languages, test-to-code density, and imported coverage from LCOV, Cobertura, JaCoCo, coverage.py, and Istanbul/NYC.

The IEEE 1045-1992 basis

IEEE Std 1045-1992 is the *IEEE Standard for Software Productivity Metrics*. It defines how to count source statements consistently so figures are comparable between tools and teams. oxide-sloc implements its **physical SLOC** model — counting physical source lines under explicit, documented rules — and exposes the standard's discretionary policies (mixed lines, continuation lines, compiler directives, blank lines inside block comments) as settings rather than baking in one interpretation. That is what lets two teams agree on a number instead of arguing about it.

> **WHY POLICIES MATTER**
>
> A line like `x = 1; // reset` contains both code and a comment. Whether it counts as code, comment, or both changes the totals. IEEE 1045 does not mandate one answer — it requires you to *state* the answer. oxide-sloc makes that answer a configurable parameter (see §6).

Three ways to run

INTERFACES

Interface	Command	Best for
CLI	`oxide-sloc analyze ./repo`	Scripting, CI pipelines, one-off scans
Web UI	`oxide-sloc serve`	Guided scans, visual reports, team access
MCP server	`sloc-mcp`	AI agents (Claude, Copilot) calling it as a tool

How it compares to cloc / tokei / scc

oxide-sloc deliberately supports fewer languages (60) than cloc, tokei, or scc (240+) and is not built to win raw line-counting throughput benchmarks. Its advantage is **analysis depth and delivery**: a web UI, HTML/PDF reports, trend history, coverage import, git hotspots, a REST API, and native AI-agent integration the others do not offer.

CAPABILITY COMPARISON

Capability	oxide-sloc	cloc	tokei	scc
Languages	60	250+	240+	240+
Web UI + HTML/PDF reports	**Yes**	No	No	No
MCP server (AI agent tools)	**Yes**	No	No	No
Test-function detection	**Yes**	No	No	No
Trend / history tracking	**Yes**	No	No	No
Coverage file import	**Yes**	No	No	No
Git Hotspots (churn × size)	**Yes**	No	No	No
COCOMO + complexity + DRYness	**Yes**	No	No	**Yes**
IEEE 1045-1992 compliance	**Yes**	partial	No	No
REST API + SVG badge	**Yes**	No	No	No
Offline / air-gapped build	**Yes**	No	No	No

SECTION 02

Installation

Pick the row that matches your platform. The fastest path on any supported system is the bootstrap script, which installs on first run and then opens the web UI.

The one-command path

```
bash scripts/run.sh   # installs on first run, then opens http://127.0.0.1:4317
```

`run.sh` auto-detects the best installation route, makes **no network calls by default**, and launches the local web UI when it finishes. What it does under the hood depends on your platform:

WHAT RUN.SH DOES PER PLATFORM

Platform	What happens
Windows 10/11 (Git Bash)	Extracts the pre-built binary from `dist/oxide-sloc-windows-x64.zip` — no Rust, no compilation
Linux — Rust installed	Builds offline from the committed `vendor.tar.xz` crate sources
Linux — no Rust, toolchain committed	Bootstraps Rust from `toolchain/` archives into `.tools/`, then builds offline
Linux — online	`bash scripts/internal/install.sh --online` downloads the release binary from GitHub
Air-gapped	Fully supported — see the air-gap matrix below

FORCE A REINSTALL

Pass `--rebuild`, `--force`, or `-f` to `run.sh` to force a fresh install even when a binary already exists — useful after a `git pull` that updates `dist/` or the vendored sources.

Package managers

PACKAGE-MANAGER INSTALLATION

Manager	Command	Platform
winget	`winget install NimaShafie.OxideSLOC`	Windows
Chocolatey	`choco install oxide-sloc`	Windows
Scoop	`scoop install oxide-sloc`	Windows
Homebrew	`brew install oxide-sloc/oxide-sloc/oxide-sloc`	macOS / Linux
Nix	`nix run github:oxide-sloc/oxide-sloc`	Linux / macOS
cargo	`cargo install oxide-sloc --locked`	Any (Rust)
DEB / RPM	Download from the Releases page	Ubuntu / RHEL

Docker

The published image at `ghcr.io/nimashafie/oxide-sloc` runs the server by default and also works as a one-shot CLI:

```
# Server (web UI) with an auth key
export SLOC_API_KEY=$(openssl rand -hex 32)
docker compose up

# CLI via Docker — analyze a host directory, read-only mount
docker run --rm -v /path/to/your/repo:/repo:ro \
  ghcr.io/nimashafie/oxide-sloc:latest analyze /repo --plain
```

Air-gapped & offline installation

Every `git clone` contains everything needed to install and run offline. No path below makes a network call unless you explicitly opt in with `--online`.

WHAT SHIPS INSIDE THE REPOSITORY

What	File	Size	Status
Windows pre-built binary	`dist/oxide-sloc-windows-x64.zip`	~9 MB	Always committed (CI-updated each release)
All Rust crate sources (~328 crates)	`vendor.tar.xz`	~35 MB	Always committed
Rust version pin	`rust-toolchain.toml`	—	Always committed
Cargo offline config	`.cargo/config.toml`	—	Written at build time
Rust compiler + cargo (Linux)	`toolchain/rust-toolchain-linux-*`	≤45 MB/part	Maintainer step (commit after bundling)

OFFLINE INSTALLATION PATHS

Path	Platform	Prereqs on target	When to use
W — Pre-built Windows binary	Windows	Git Bash, `unzip`	Default Windows path — no Rust
A — Bundled toolchain build	Linux	`bash`, `tar`, `gcc`	No Rust, no internet (toolchain committed)
B — Vendor source build	Linux / Windows	Rust ≥1.95	Rust already installed
C — Airgap kit	Linux	`bash`, `tar`, `sha256sum`	No Rust, fully self-contained static binary
D — Download from GitHub	Linux	`bash`, `tar`, `curl`	No Rust, but has internet

Path A — what install.sh does with a committed toolchain

1. Detects no `cargo` on `PATH`.
2. Finds `toolchain/rust-toolchain-linux-{arch}.tar.gz`.
3. Verifies its SHA-256 against `toolchain/checksums.sha256`.
4. Extracts to `.tools/` and exports `RUSTUP_HOME` / `CARGO_HOME` / `PATH` for the session — no root, no system-wide change.
5. Decompresses `vendor.tar.xz` (~35 MB → ~362 MB) once.
6. Runs `cargo build --release --offline -p oxide-sloc`.

7. Copies the binary to `oxide-sloc` in the repo root.

> **MAINTAINER STEP**
>
> If `toolchain/` is absent on a fresh clone, a maintainer must populate it once from a machine with internet: `bash scripts/internal/bundle-rust-toolchain.sh linux-x86_64` (or `linux-arm64`), then commit `toolchain/`. The script reads the version from `rust-toolchain.toml`, downloads the matching installer, verifies the upstream SHA-256, and splits the archive into ≤45 MB parts.

Path C — the self-contained airgap kit

Produces a single ~700 MB archive bundling the Rust host toolchain, the musl C toolchain, vendored crate sources, and the full source tree — yielding a **fully static binary** you can copy anywhere on Linux with no runtime dependencies.

```
# On a build machine with internet:
bash scripts/internal/make-airgap-kit.sh linux-x86_64
# → oxide-sloc-airgap-kit-linux-x86_64-v{version}.tar.gz

# On the air-gapped target:
tar xzf oxide-sloc-airgap-kit-linux-x86_64-v*.tar.gz
cd oxide-sloc-airgap-kit-*/
bash install.sh
```

Native desktop file dialogs (optional)

The default build is headless and needs no GUI libraries. To enable the native OS file picker on a Linux desktop, build with the feature flag and install the system packages first:

```
cargo build --release --offline --features native-dialog
# Debian/Ubuntu:  apt install libwayland-dev libgtk-3-dev libxdo-dev
# RHEL 8/9:       dnf install wayland-devel gtk3-devel libxdo-devel
```

> **RUNTIME NETWORK REQUIREMENTS**
>
> The web UI, `analyze`, `report`, and PDF export need **no network** (PDF uses a locally installed Chromium). Only email delivery (`--smtp-to`) and webhook delivery (`--webhook-url`) require outbound access.

SECTION 03

Quick start

Two routes get you to a first result: the web UI for a guided, visual scan, or the CLI for an immediate terminal summary.

Option 1 — the web UI

1. **Launch.** Run `bash scripts/run.sh` (first time installs automatically) or, once installed, `oxide-sloc serve`. Your browser opens to `http://127.0.0.1:4317`.
2. **Quick Scan.** On the scan page, pick a project folder and press `Quick Scan` — it submits straight from step 1 with all defaults applied.
3. **Read the report.** You get SLOC totals, a per-language breakdown, the COCOMO estimate, complexity, and (against a git repo) a ranked Git Hotspots table.
4. **Export.** Download the report as HTML, PDF, CSV, or a 4-sheet Excel workbook.

Option 2 — the CLI

```
# Colored terminal summary
oxide-sloc analyze ./my-repo

# Everything at once: JSON, HTML, CSV, Excel
oxide-sloc analyze ./my-repo -j out.json -H out.html -c out.csv -x out.xlsx
```

Create a config file

```
oxide-sloc init    # writes .oxide-sloc.toml with all options documented inline
```

CLI flags override values in the config file, which override built-in defaults. The full key reference is in §6.

SECTION 04

CLI reference

One binary, six commands. Every command accepts `--help` for its full, authoritative flag list; this section documents what each does and the flags you will actually reach for.

COMMANDS AT A GLANCE

Command	Purpose
`analyze`	Scan a path and produce metrics + reports
`report`	Re-render reports from a saved JSON result — no re-scan
`diff`	Compute the delta between two JSON results
`serve`	Start the web UI / API server
`send`	Deliver a result by SMTP email or webhook
`init`	Generate a documented `.oxide-sloc.toml`

analyze — the workhorse

Scans one or more paths, computes all metrics, and emits any requested output formats.

```
oxide-sloc analyze ./my-repo                          # colored summary
oxide-sloc analyze ./my-repo --plain                  # key=value, CI-friendly
oxide-sloc analyze ./my-repo -j out.json -H out.html -c out.csv -x out.xlsx
oxide-sloc analyze ./my-repo --per-file               # include per-file rows
oxide-sloc analyze ./my-repo --fail-on-warnings --fail-below 10000
oxide-sloc analyze ./my-repo --enabled-language rust --enabled-language python
oxide-sloc analyze ./my-repo --submodule-breakdown
oxide-sloc analyze ./my-repo --per-file --activity-window 90   # Git Hotspots
```

Output flags

Flag	Short	Effect
`--json-out <file>`	`-j`	Machine-readable JSON result (stable across versions)
`--html-out <file>`	`-H`	Self-contained single-file HTML report
`--csv-out <file>`	`-c`	Multi-section CSV (Summary, By Language, Per File, Skipped)
`--xlsx-out <file>`	`-x`	Four-sheet Excel workbook (same sections)
`--pdf-out <file>`	—	PDF report — pure-Rust generation, no browser required
`--plain`	—	Machine-friendly `key=value` terminal output
`--per-file`	—	Emit per-file rows in addition to aggregates
`--report-title <str>`	—	Title embedded in HTML and PDF reports

Scope & discovery flags

Flag	Effect
`--enabled-language <lang>`	Restrict to one language; repeat for several
`--include-glob <pat>`	Only scan paths matching the glob, e.g. `"src/**/*.py"`
`--exclude-glob <pat>`	Skip paths matching the glob, e.g. `"vendor/**"`
`--no-ignore-files`	Ignore `.gitignore` / `.slocignore` and scan everything
`--follow-symlinks`	Follow symbolic links during discovery (off by default)
`--submodule-breakdown`	Detect `.gitmodules` and emit per-submodule stats
`--activity-window <days>`	Git Hotspots window in days (default 90; `0` disables). Needs a git checkout

Counting-policy flags (IEEE 1045)

Flag	Effect
`--mixed-line-policy <p>`	How a line with both code and a comment is classified
`--continuation-line-policy <p>`	How wrapped / continued logical lines are counted
`--blank-in-block-comment-policy <p>`	How blank lines inside block comments are classified
`--count-compiler-directives`	Whether preprocessor / compiler directives count as code
`--docstrings-as-code`	Count Python triple-quoted docstrings as code, not comments

CI gate flags

Flag	Effect
`--fail-on-warnings`	Exit non-zero if the scan produced warnings (e.g. binary files found)
`--fail-below <n>`	Exit non-zero if total code lines fall below `n`
`--config <file>`	Load a specific TOML config / CI preset

report — re-render without re-scanning

Takes a saved JSON result and produces any other format. Because it never re-reads the source tree, it is fast and deterministic — ideal for regenerating a PDF from an archived scan.

```
oxide-sloc report result.json -H report.html --pdf-out report.pdf
```

diff — compare two scans

Computes the change between a baseline and a current result: added/removed files, per-language deltas, and net line changes. Useful as a pull-request gate or release note.

```
oxide-sloc diff baseline.json current.json -j delta.json
```

serve — the web UI & API

```
oxide-sloc serve                    # local: http://127.0.0.1:4317, opens browser
oxide-sloc serve --server           # LAN: binds 0.0.0.0:4317, no auto-open
oxide-sloc serve --server --config /etc/oxide-sloc/config.toml
```

send — deliver a result

```
# Email via SMTP
oxide-sloc send result.json --smtp-to [email protected] --smtp-host smtp.example.com

# Webhook (Slack, Teams, custom)
oxide-sloc send result.json --webhook-url "https://hooks.slack.com/services/ … "
```

Prefer the environment variables `SLOC_SMTP_HOST` / `SLOC_SMTP_USER` / `SLOC_SMTP_PASS` and `SLOC_WEBHOOK_TOKEN` over passing secrets on the command line, which exposes them in process listings.

init — scaffold a config

```
oxide-sloc init    # creates .oxide-sloc.toml with every option documented inline
```

> **DISCOVERABILITY**
>
> Run `oxide-sloc <command> --help` for the complete, version-accurate flag list of any command — it is generated from the binary itself and is always authoritative.

SECTION 05

Web UI & the scan flow

Start the server with `oxide-sloc serve` and open `http://127.0.0.1:4317`. The home page is a navigation hub; the heart of the app is the guided four-step scan flow.

The pages

WEB UI PAGES

Page	Route	What it does
Home	`/`	Quick-launch cards for every feature
Scan	`/` (flow)	The guided 4-step scan; **Quick Scan** submits with defaults
View Reports	`/reports`	Browse and re-open saved reports
Test Metrics	`/test-metrics`	Test-function counts, density, coverage import
Trend Reports	`/trend-reports`	Metrics over time for a project root
Compare Scans	`/compare-scans`	Delta between any two saved runs
Git Browser	`/git-browser`	Scan any branch, tag, or commit

The four steps

1. **Select project.** Choose the directory to analyze. In local mode this is a native OS folder dialog; in server mode it is a browser directory upload (see §10). `Quick Scan` here skips straight to the result with all defaults.
2. **Counting rules.** Set the IEEE 1045 policies and discovery options — mixed-line handling, continuation lines, compiler directives, docstrings-as-code, ignore-file behaviour, symlink following, language filters, and include/exclude globs. Every control maps to a config key in §6.
3. **Outputs.** Choose which artifacts to produce — HTML, PDF, CSV, Excel, JSON — set the report title, and toggle features such as per-file rows, submodule breakdown, and the Git Hotspots activity window.
4. **Review & run.** Confirm the assembled configuration and execute. The result opens as a full report with SLOC totals, per-language breakdown, complexity, COCOMO, and (against a git repo) the ranked Git Hotspots table.

Reports, trends & comparison

Every run is saved to a scan registry, so reports persist between sessions and feed the trend and comparison views. The **Compare Scans** page is the visual equivalent of `diff`; **Trend Reports** plots a project's metrics across its scan history; **Git Browser** lets you scan any branch, tag, or commit without checking it out manually.

> **SCREENSHOTS**
>
> Annotated screenshots of every page live in the repository under `docs/screenshots/` (`home-page.png`, `view-reports.png`, `compare-delta.png`, `trend-reports.png`, `test-metrics.png`). They are kept out of this manual to keep it fast and lightweight.

SECTION 06

Configuration reference

Run `oxide-sloc init` to generate `.oxide-sloc.toml` with every option documented inline. This section catalogues every parameter you can set, the values it accepts, and what it does.

> **PRECEDENCE**
>
> **CLI flag** > **config file** (`--config` or a discovered `.oxide-sloc.toml`) > **built-in default**. The web UI's "Counting rules" and "Outputs" steps set exactly these same parameters.

IEEE 1045 counting policies

`mixed_line_policy` enum

`default: code-only`

Governs a physical line containing *both* code and a comment, e.g. `total += 1; // accumulate`.

- `code-only` — the line counts once, as **code**. The inline comment does not add a comment line. (Default; mirrors the web UI.)
- `code-and-comment` — the line is counted in **both** the code and comment totals.
- `comment-only` — the line counts as a **comment** only (rare; for comment-density studies).

`continuation_line_policy` enum

Governs how a single logical statement spread across several physical lines (line continuations, wrapped expressions, multi-line argument lists) is counted.

- **Per-physical-line** — each physical line of the continuation is counted (true physical SLOC).
- **Per-logical-statement** — the whole continued statement counts once, collapsing wrapped lines into a single logical SLOC.

`blank_in_block_comment_policy` enum

Governs a blank line that falls *inside* a multi-line block comment.

- **As comment** — it counts toward the comment total (part of the comment block).
- **As blank** — it counts toward the blank-line total instead.

`count_compiler_directives` `boolean`

Whether preprocessor / compiler directives — `#include`, `#define`, `#pragma`, `#if`, and language equivalents — are counted as **code**. Enable to treat directives as source; disable to exclude build-time scaffolding from the code total.

`docstrings_as_code` `boolean` `default: false`

When true, Python triple-quoted docstrings are counted as **code** rather than comments. Leave false to treat docstrings as documentation (the conventional reading).

Discovery & scope

`enabled_languages` `list<string>` `default: all`

Restrict the scan to specific languages by identifier (e.g. `["rust", "python"]`). Empty means all 60 supported languages. CLI: repeated `--enabled-language`.

`include_globs` `list<glob>` `default: all`

Only paths matching these glob patterns are scanned, e.g. `["src/**/*.py"]`. CLI: `--include-glob`.

`exclude_globs` `list<glob>` `default: none`

Paths matching these globs are skipped, e.g. `["vendor/**", "**/generated/**"]`. CLI: `--exclude-glob`.

`no_ignore_files` `boolean` `default: false`

When true, `.gitignore` and `.slocignore` rules are ignored and otherwise-excluded files are scanned. Use for full-scope audits.

`follow_symlinks` `boolean` `default: false`

Follow symbolic links during file discovery. Off by default to avoid double-counting and link loops.

`submodule_breakdown` `boolean` `default: true (CI)`

Detect `.gitmodules` and emit per-submodule statistics in the report. CLI: `--submodule-breakdown`.

activity_window integer (days) — default: 90

The Git Hotspots look-back window. Files are ranked by code lines × commits within this many days. Set `0` to disable hotspots. Requires the scan path to be a git checkout. CLI: `--activity-window`.

Output & CI gates

report_title string

Title embedded in HTML and PDF reports. CLI: `--report-title`.

per_file boolean — default: false

Include per-file rows (and the "Per File" CSV/Excel sheet) in addition to aggregates. CLI: `--per-file`.

fail_on_warnings boolean — default: false

Exit with a non-zero status if the scan produced warnings (for example, binary files encountered). Turns advisory conditions into pipeline failures.

fail_below integer — default: unset

Exit non-zero if total code lines fall below this threshold — a guard against accidental empty or mis-scoped scans in CI.

Shipped CI presets

Ready-made config files in `ci/` cover common postures. Select one with `--config` or the Jenkins `CI_PRESET` parameter.

CI CONFIGURATION PRESETS

Preset file	Posture
`ci/sloc-ci-default.toml`	Balanced defaults — mirrors the web UI out of the box
`ci/sloc-ci-strict.toml`	Fail-fast — the pipeline errors if binary files are found
`ci/sloc-ci-full-scope.toml`	Audit mode — counts everything, including vendored code

NOT COUNTED BY DESIGN

TOML, Markdown, and YAML are intentionally unsupported — there is no meaningful SLOC metric for them — so they never appear in code totals regardless of policy.

SECTION 07

Metrics glossary

Every number in a report, explained in depth: what it measures, how oxide-sloc derives it, and how to read it. Where a metric is an approximation, that is stated plainly.

Line classification

Code lines SLOC

Physical source lines that contain executable or declarative code. This is the headline number and the basis for COCOMO, complexity, and hotspot ranking. Counted per IEEE 1045-1992 physical-SLOC rules under your configured policies.

Comment lines

Lines that are wholly comment, plus — depending on `mixed_line_policy` — the comment portion of mixed lines, and — depending on `blank_in_block_comment_policy` — blank lines inside block comments.

Blank lines

Empty or whitespace-only lines. A measure of visual spacing, not size; excluded from the code total.

Comment density

Derived ratio of comment lines to code lines, a rough proxy for how documented the source is.

```
comment density = comment lines / code lines
```

Symbol counts

Lexically detected declarations, per language — pattern-based, not a full parse, so fast and approximate.

Symbol	What is counted
Functions	Function / method / procedure definitions
Classes	Classes, structs, enums, traits, interfaces — type definitions
Variables	Top-level and notable variable / constant declarations
Imports	`import` / `use` / `#include` / `require` statements

ULOC & DRYness

Unique lines of code `ULOC`

The number of *distinct* code lines after removing exact duplicates. Comparing ULOC to total code lines reveals copy-paste and boilerplate.

DRYness

"Don't Repeat Yourself"-ness — the share of code lines that are unique. A value near 100% means little repetition; a low value flags heavy duplication.

```
DRYness = ULOC / code lines × 100%
```

Duplicate-file detection

Files whose content is byte-for-byte (or near) identical are flagged on every run, surfacing accidental copies and vendored duplicates.

Cyclomatic complexity (approximate)

Approximate cyclomatic complexity `CC`

A lexical approximation of McCabe cyclomatic complexity: a count of branch / decision keywords (`if`, `for`, `while`, `case`, `catch`, `&&`, `||`, …) summed per code line.

Important: this is *not* a true control-flow-graph McCabe computation (`M = E − N + 2P` over a CFG). It is a keyword tally that correlates with branching density and is fast across 60 languages. Treat it as a relative signal — comparing files or trends — not an exact decision-count.

COCOMO I — effort & cost

Constructive Cost Model COCOMO

Barry Boehm's 1981 algorithmic estimation model. oxide-sloc applies the **Basic COCOMO I, organic mode** equations, driving them from the measured code-line count (as KLOC — thousands of lines of code) to produce an order-of-magnitude effort, schedule, and team-size estimate.

```
KLOC   = code lines / 1000
Effort = a × KLOC^b          (person-months, PM)
Time   = c × Effort^d        (calendar months, TDEV)
People = Effort / Time           (average staff)
```

For organic mode oxide-sloc uses Boehm's canonical coefficients `a=2.4 b=1.05 c=2.5 d=0.38` and converts effort to a currency cost with an assumed loaded monthly rate.

How to read it. COCOMO answers "if this many lines were written from scratch under typical conditions, roughly what would it have cost?" It is an estimation model, not a measurement of what your project actually spent — use it for relative scale and rough budgeting, not invoicing.

COCOMO OUTPUTS

Output	Unit	Meaning
Effort	person-months (PM)	Total development labour
Schedule (TDEV)	calendar months	Nominal time to develop
Team size	persons	Effort ÷ schedule
Estimated cost	currency	Effort × loaded monthly rate

BASIC COCOMO COEFFICIENTS BY PROJECT CLASS

Class	a	b	c	d	Typical project
Organic	2.4	1.05	2.5	0.38	Small team, familiar, well-understood
Semi-detached	3.0	1.12	2.5	0.35	Medium size, mixed experience
Embedded	3.6	1.20	2.5	0.32	Tight constraints, complex, high reliability

Git Hotspots

Git Hotspots

A refactoring-priority ranking. From a single `git log` pass, oxide-sloc reads each file's commit count and last-changed date over the activity window (90 days by default), then ranks files by the product of size and churn:

```
hotspot score = code lines × commits in window
```

The premise (from Adam Tornhill's "hotspots" technique): a file that is both **large** and **frequently changed** is where complexity and change-risk concentrate — the best refactoring candidate. A large file nobody touches is stable; a tiny file changed daily is cheap to fix. The intersection is where attention pays off.

Column	Meaning
Code lines	Size of the file
Commits	Number of commits touching it within the window
Last changed	Date of the most recent commit to it
Score	Code lines × commits — the rank key

Test metrics

Test-function detection

Lexical detection of test functions across all 60 languages — recognising framework conventions (e.g. `#[test]`, `def test_*`, `@Test`, `it( ... )`, `func TestXxx`). Pattern-based and fast rather than semantic analysis.

Test-to-code density

The ratio of test code to production code — a quick read on how heavily a codebase is exercised by its own tests.

`test density = test lines / code lines`

Coverage import

oxide-sloc does not run your tests; it *imports* coverage your test runner already produced, in five formats, and folds the percentages into the report.

SUPPORTED COVERAGE FORMATS

Format	Typical source
LCOV (`lcov.info`)	gcov, llvm-cov, many JS tools
Cobertura XML	Java, Python (pytest-cov), .NET
JaCoCo XML	JVM ecosystems
coverage.py JSON	Python
Istanbul / NYC JSON	JavaScript / TypeScript

SECTION 08

Output formats

One analysis pass can emit five artifacts. All are produced from the same in-memory result, so they always agree; JSON is the canonical, re-renderable source of truth.

OUTPUT ARTIFACTS

Format	Flag	Notes
JSON	`-j / --json-out`	Machine-readable, stable across versions; feeds dashboards and re-rendering
HTML	`-H / --html-out`	Self-contained single file (charts compiled in, no CDN calls)
PDF	`--pdf-out`	Pure-Rust generation — no browser or external tool required on the agent
CSV	`-c / --csv-out`	Multi-section: Summary, By Language, Per File, Skipped Files
Excel	`-x / --xlsx-out`	Four-sheet workbook mirroring the CSV sections

Re-rendering from JSON

```
oxide-sloc report result.json -H report.html --pdf-out report.pdf
```

The JSON shape

Path	Contents
`summary_totals.code_lines`	Total code lines (the headline metric)
`summary_totals.comment_lines`	Total comment lines
`summary_totals.blank_lines`	Total blank lines
`summary_totals.files_analyzed`	File count
`totals_by_language[]`	Per-language rows: `language.display_name`, `files`, `code_lines`, `comment_lines`, `blank_lines`

```
# Read the headline number with python3
CODE_LINES=$(python3 -c "import json;print(json.load(open('result.json'))['summary_totals']['
```

Standalone downloads via the API

```
GET /runs/csv/<run_id>
GET /runs/xlsx/<run_id>
```

SECTION 09

Metrics REST API

A running server exposes a small, stable HTTP API for dashboards, badges, embeds, and automation. The full contract is published as OpenAPI 3.1 at `GET /api/openapi.yaml` (also committed at `docs/openapi.yaml`).

Endpoints

METRICS & DATA API

Endpoint	Returns
`GET /api/metrics/latest`	Metrics for the most recent scan
`GET /api/metrics/:run_id`	Metrics for a specific run
`GET /api/project-history?path=<dir>`	Scan history for a project root (feeds trends)
`GET /runs/csv/:run_id`	CSV download for a run
`GET /runs/xlsx/:run_id`	Excel download for a run
`GET /api/openapi.yaml`	The OpenAPI 3.1 specification

Badges & embeds

Endpoint	Returns
`GET /badge/:metric`	SVG badge — `code-lines`, `files`, `comment-lines`, or `blank-lines`
`GET /embed/summary`	Embeddable HTML summary widget

```
# Embed a live badge in a README
![Code Lines](http://your-host:4317/badge/code-lines)
```

Health & version

Endpoint	Use
`GET /healthz`	Returns `200 ok` — used by the Docker `HEALTHCHECK`
`GET /api/health`	Conventional REST health path for monitoring tools
`GET /api/version`	Returns `{"name":"oxide-sloc","version":"X.Y.Z"}`

AUTH & LIMITS

When `SLOC_API_KEY` is set, every request must carry `Authorization: Bearer <key>` or `X-API-Key: <key>`. A sliding-window rate limiter allows **60 requests per 60-second window per client IP**; excess requests get `HTTP 429`. A running server also serves `/llms.txt` and `/llms-full.txt` for AI-agent self-discovery.

SECTION 10

Local vs server mode

The same `serve` command runs two ways. Local mode is for one person on one machine; server mode (`--server`) is for hosting oxide-sloc so other users can reach it over a network.

BEHAVIOURAL DIFFERENCES

	Local (default)	Server (`--server`)
Bind address	`127.0.0.1:4317`	`0.0.0.0:4317`
Browser auto-open	Yes	No
File picker	Native OS dialog	Browser directory upload
OS path opener	Yes (Explorer/Finder)	Toast message (not available)
Startup message	"local web UI"	"server"

In server mode the native picker is replaced by a browser folder-picker: users click **Browse**, select a folder on their own machine, and the files are uploaded to the server's temp directory. The path field auto-fills with the server-side temp path, and the uploaded files are deleted automatically once the scan completes. Because the OS path opener cannot open a window on a remote client, clicking it shows an informational message instead.

> **SHARED STATE**
>
> In server mode the scan registry and report artifacts are shared across all sessions — every user sees the same saved reports, trends, and comparisons.

SECTION 11

LAN server deployment

For personal use, `bash scripts/run.sh` is all you need. This section is for hosting oxide-sloc persistently so a team can reach it — the quick LAN launcher, Docker, systemd, reverse proxies, TLS, authentication, and operational endpoints.

Quick LAN launch

```
bash scripts/serve-server.sh              # opens to the LAN, prints every reachable address
bash scripts/serve-server.sh --with-auth  # also generates a session key and requires login
```

`serve-server.sh` does not auto-install — run `scripts/run.sh` at least once first so the binary exists.

Option A — Docker Compose (recommended)

```
docker compose up -d        # build & start; survives reboots via restart: unless-stopped
docker compose logs -f      # tail logs
docker compose down         # stop
```

The container runs with `--server` by default. To let users analyze a host directory, bind-mount it read-only and have them enter the target path (e.g. `/repo`) in the form:

```
volumes:
  - type: bind
    source: /path/to/project
    target: /repo
    read_only: true
```

The image ships a `HEALTHCHECK` that polls `/healthz` every 30 seconds. In Docker, set `SLOC_BROWSER_NOSANDBOX=1` so the bundled Chromium can run for PDF export.

Option B — systemd (bare-metal / VPS)

1. **Install the binary.** From a release archive (`sudo install -m 755 oxide-sloc /usr/local/bin/`) or from source (`cargo build --release -p oxide-sloc`).
2. **Create a service user.** `sudo useradd --system --no-create-home --shell /usr/sbin/nologin oxide-sloc` and a working dir under `/opt/oxide-sloc`.

3. **Install a config (optional).** Copy `deploy/server.toml` to `/etc/oxide-sloc/config.toml` and reference it with `--config` in `ExecStart`.
4. **Enable the service.** Easiest: `sudo bash scripts/internal/install-systemd.sh` (undo with `--uninstall`). Manually: copy `deploy/oxide-sloc.service`, `systemctl daemon-reload`, then `systemctl enable --now oxide-sloc`.

```
sudo systemctl status oxide-sloc
sudo journalctl -u oxide-sloc -f
```

Reverse proxy & TLS

For HTTPS or a custom domain, put Nginx or Caddy in front and bind oxide-sloc to loopback (`bind_address = "127.0.0.1:4317"` in `server.toml`) so the port is not directly internet-facing:

```
server {
    listen 443 ssl;
    server_name sloc.example.com;
    # ssl_certificate / ssl_certificate_key ...
    location / {
        proxy_pass http://127.0.0.1:4317;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

Alternatively, oxide-sloc can terminate TLS itself with PEM files — set `SLOC_TLS_CERT` and `SLOC_TLS_KEY`. When both are set the server reports `running at https:// ... (TLS)`. A reverse proxy is preferred for production because it keeps certificate management out of the binary.

Security controls

SERVER SECURITY

Control	How
Authentication	Set `SLOC_API_KEY`; clients send `Authorization: Bearer <key>` or `X-API-Key: <key>`. Running `--server` without it logs a startup warning.
TLS	`SLOC_TLS_CERT` + `SLOC_TLS_KEY` (native) or terminate at a proxy. Cleartext `--server` logs a warning.
Rate limiting	Sliding window: 60 requests / 60 s per client IP across all routes; excess gets `HTTP 429`.
Path allow-list	`SLOC_ALLOWED_ROOTS` restricts which host directories may be analyzed.

Server environment variables

Variable	Purpose	Default
`OXIDE_SLOC_ROOT`	Working dir for outputs & relative path resolution	current dir
`SLOC_BROWSER`	Path to a Chromium-based browser for PDF export	auto-detected
`SLOC_BROWSER_NOSANDBOX`	Set `1` to add `--no-sandbox` (required in Docker)	unset
`SLOC_REGISTRY_PATH`	Override path for `registry.json`	`<out>/registry.json`
`SLOC_API_KEY`	Bearer token for request auth	unset (no auth)
`SLOC_TLS_CERT` / `SLOC_TLS_KEY`	PEM cert / key for native TLS	unset
`RUST_LOG`	Log level: error/warn/info/debug/trace	`info`

SECTION 12

CI/CD integration

oxide-sloc is a single self-contained binary with no daemons or runtime dependencies, so every CI integration follows one pattern: **acquire the binary → run the scan → consume the outputs**. Ready-made pipelines ship for the major platforms.

SHIPPED CI CONFIGURATIONS

Platform	File
GitHub Actions	`ci/sloc-github-action.yml` (copy to `.github/workflows/`)
GitHub Marketplace	`uses: NimaShafie/oxide-sloc@main` (`action.yml`)
Jenkins	`Jenkinsfile` at repo root; helpers under `ci/jenkins/`
GitLab CI	`.gitlab-ci.yml` at repo root
Bitbucket Pipelines	`examples/bitbucket/bitbucket-pipelines.yml`
Azure Pipelines	`examples/azure/azure-pipelines.yml`

GitHub Actions — Marketplace action

```
- uses: NimaShafie/oxide-sloc@main
  id: sloc
  with:
    path: .
    html-out: sloc-out/report.html
- run: echo "Code lines ${{ steps.sloc.outputs.code-lines }}"
```

Or add a scan step to an existing workflow by decompressing the vendored sources, installing, and running:

```
- name: Decompress vendor sources
  run: tar -xJf vendor.tar.xz
- name: Install oxide-sloc
  run: cargo install --path crates/sloc-cli
- name: Run SLOC scan
  run: |
    oxide-sloc analyze ./src \
      --json-out out/result.json --html-out out/report.html \
      --report-title "SLOC - ${{ github.ref_name }}"
- uses: actions/upload-artifact@v4
  with:
    name: sloc-report
    path: out/
```

The repository's own workflows are `ci.yml` (fmt → lint → build → smoke → web health-check on every push/PR), `release.yml` (cross-compile 5 platforms → GPG + cosign sign → VirusTotal scan → publish on `v*` tags), and `vt-scan.yml` (manual VirusTotal scan).

Jenkins

The root `Jenkinsfile` is a fully parameterized pipeline: setup, quality gates, analysis, web health-check, optional webhook/email delivery, and artifact publishing with build-over-build trend charts. Create the job via the GUI (*Pipeline script from SCM*) or bootstrap it with `ci/jenkins/job-config.xml`. Run `ci/jenkins/preflight.sh` first — every line must print `[ok]`.

SELECTED JENKINS BUILD PARAMETERS

Parameter	Default	Meaning
`SCAN_PATH`	`tests/fixtures/basic`	Directory / paths to scan
`CI_PRESET`	`default`	`default` / `none` / `strict` / `full-scope`
`MIXED_LINE_POLICY`	`code-only`	Inline-comment line classification
`ACTIVITY_WINDOW`	`90`	Git Hotspots window in days (`0` = off)
`DOCSTRINGS_AS_CODE`	`false`	Count Python docstrings as code
`SUBMODULE_BREAKDOWN`	`true`	Per-submodule stats
`FOLLOW_SYMLINKS` / `NO_IGNORE_FILES`	`false`	Discovery overrides
`ENABLED_LANGUAGES`	*all*	Comma-separated language filter
`INCLUDE_GLOBS` / `EXCLUDE_GLOBS`	*all / none*	Comma-separated glob scope
`GENERATE_HTML` / `GENERATE_PDF`	`true`	Which reports to write
`WEBHOOK_URL` / `EMAIL_RECIPIENTS`	*skip*	Optional delivery targets
`ARTIFACT_REPO_TYPE`	`none`	Artifact backend (see below)

JSON IS ALWAYS GENERATED

Regardless of parameters, `result.json` is always produced — it drives the trend plots, the build-description summary, and the `send` delivery subcommand.

Artifact repository publishing

The script `ci/artifact-push.sh` uploads scan artifacts to an external repository, driven by `ARTIFACT_REPO_*` environment variables. Set `ARTIFACT_DRY_RUN=true` to preview without network calls, and `ARTIFACT_GENERATE_MANIFEST=true` to upload a `checksums.sha256` alongside.

SUPPORTED ARTIFACT BACKENDS (ARTIFACT_REPO_TYPE)

Type	Backend
`artifactory`	JFrog Artifactory (REST PUT, Basic or API-key auth)
`nexus` / `nexus2`	Sonatype Nexus 3 (raw repo) / Nexus 2 content API
`s3`	Amazon S3 (`aws s3 cp`)
`minio`	MinIO (S3 with custom `--endpoint-url`)
`azure-blob`	Azure Blob Storage (`az storage blob upload`)
`generic-http`	Any HTTP server accepting PUT (Basic or Bearer auth)

Credentials come from the Jenkins Secret-Text credentials `SLOC_ARTIFACT_REPO_USER` / `SLOC_ARTIFACT_REPO_PASS` (or matching env vars). Provider specifics go in `ARTIFACT_REPO_EXTRA`. GitLab's `.gitlab-ci.yml` (stages `quality → build → smoke → archive`) and Bitbucket's pipeline both include a *Publish to Nexus* step, off by default.

SECTION 13

AI / MCP integration

The `sloc-mcp` binary implements the Model Context Protocol, making oxide-sloc callable as a tool from Claude Desktop, Claude Code, and any MCP-compatible host — so an agent can scan a path and reason over the metrics directly.

```
cargo build -p sloc-mcp
```

Register it with Claude Code via a project `.mcp.json`:

```
{ "mcpServers": { "oxide-sloc": { "command": "sloc-mcp" } } }
```

Available MCP tools

Tool	Purpose
`analyze_path`	Scan a directory and return metrics
`get_metrics_latest`	Metrics for the most recent run
`get_run_metrics`	Metrics for a specific run id
`get_metrics_history`	History for a project root
`compare_runs`	Delta between two runs
`ingest_result`	Import an existing result JSON
`health_check`	Server liveness probe

Pre-built tool definitions for the Claude API (`tool_use`) and OpenAI (`function_calling`) live under `docs/mcp/`. A running web server additionally serves `/llms.txt` and `/llms-full.txt` for agent self-discovery (see §9).

SECTION 14

Git integration

oxide-sloc reads git directly to power hotspots, history, submodule breakdowns, and the branch/commit browser — and can receive push events from the major forges.

What git unlocks

- **Git Hotspots** — per-file commit counts and last-changed dates from one `git log` pass over the activity window (§7).
- **Git Browser** (`/git-browser`) — scan any branch, tag, or commit without checking it out manually.
- **Submodule breakdown** — detects `.gitmodules` and reports per-submodule statistics (`--submodule-breakdown`).
- **Trend history** — successive scans of a project root build the data behind `/trend-reports` and `/api/project-history`.

Webhooks & polling

The server accepts GitHub, GitLab, and Bitbucket webhooks and can poll repositories on a schedule, so a scan can run automatically on every push. Combined with the `send` command, a result can then be delivered onward by webhook (Slack/Teams) or SMTP email.

> **NETWORK NOTE**
> Webhook and email *delivery* require outbound network access; everything else — including receiving webhooks on the LAN and all analysis — works fully offline.

SECTION 15

Maintenance & operations

Day-two tasks: keeping the offline bundles current, managing the scan registry, reading logs, and the developer build loop.

Updating

UPDATE ROUTES

Install method	Update
Repo checkout	`git pull`, then `bash scripts/run.sh --rebuild`
Package manager	The manager's own upgrade (e.g. `brew upgrade oxide-sloc`, `cargo install oxide-sloc --locked --force`)
Docker	`docker compose pull && docker compose up -d`
systemd	Install the new binary, then `systemctl restart oxide-sloc`

Maintaining the offline bundles

Script (scripts/internal/)	When to run
`update-vendor.sh`	After any dependency change — regenerates `vendor.tar.xz` + `.sha256`
`bundle-rust-toolchain.sh`	After a `rust-toolchain.toml` bump — re-bundles the Linux toolchain
`make-airgap-kit.sh`	To produce a fresh self-contained Linux airgap kit (§2, Path C)
`install-hooks.sh`	Developer setup — installs the git pre-commit hooks

The Windows binary in `dist/` is refreshed automatically by the `update-dist.yml` CI workflow after every release; no manual step is needed.

The scan registry & artifacts

Saved runs live in `registry.json` (override its location with `SLOC_REGISTRY_PATH`) alongside the generated report files in the output directory (`OXIDE_SLOC_ROOT`). To back up history, snapshot the output directory and the registry together. To reset, stop the server and remove them — reports and trends start fresh.

Logs & health

Set verbosity with `RUST_LOG` (`info` default; `debug` / `trace` for diagnosis). Under systemd, follow logs with `journalctl -u oxide-sloc -f`; under Docker, `docker compose logs -f`. Poll `/healthz`, `/api/health`, or `/api/version` for liveness and the running version.

Developer build loop

```
cargo fmt --all -- --check
cargo clippy --workspace --all-targets -- -D warnings
cargo build --workspace
cargo test --workspace
cargo run -p oxide-sloc -- serve    # http://127.0.0.1:4317
```

SECTION 16

Troubleshooting

Common failure modes and their fixes.

SYMPTOMS & FIXES

Symptom	Likely cause & fix
PDF export fails or hangs	No Chromium found, or sandbox blocked. Set `SLOC_BROWSER` to a Chromium binary; in Docker set `SLOC_BROWSER_NOSANDBOX=1`. (CLI `--pdf-out` uses pure-Rust generation and needs no browser.)
"cleartext" warning on server start	Running `--server` without TLS. Add `SLOC_TLS_CERT` / `SLOC_TLS_KEY` or terminate TLS at a reverse proxy.
"no API key" warning on server start	Running `--server` without `SLOC_API_KEY`. Set one to require bearer-token auth.
`HTTP 429 Too Many Requests`	Rate limit hit (60 req / 60 s per IP). Back off, or front the server with a proxy that pools clients.
Git Hotspots table is empty	The scan path is not a git checkout, or `activity_window` is `0`. Point at a repo and set a non-zero window.
A language is missing from totals	It may be intentionally unsupported (TOML, Markdown, YAML), filtered by `enabled_languages`, or excluded by a glob / ignore file (try `--no-ignore-files`).
Code total lower than expected	Files excluded by `.gitignore` / `.slocignore`, or a restrictive `include_globs`. Use `--per-file` to see what was counted and the "Skipped Files" sheet for what was not.
Offline Linux build can't find Rust	`toolchain/` not committed. Use `--online` (Path D), the airgap kit (Path C), or have a maintainer run `bundle-rust-toolchain.sh` and commit.
`vendor.tar.xz` checksum mismatch	Corrupted/partial checkout. Re-clone or re-fetch; the build verifies against `vendor.tar.xz.sha256` before extracting.
Server file picker won't open a folder	Expected in `--server` mode — the OS path opener can't reach a remote client. Use Browse (directory upload) instead.

SECTION 17

Acronyms & glossary

Every abbreviation and term used in this manual and in oxide-sloc's reports, expanded.

SLOC	Source Lines Of Code — the count of physical source lines, classified into code, comment, and blank.
LOC	Lines Of Code — generic term; here usually physical lines.
KLOC	Thousands (Kilo) of Lines Of Code — `code_lines / 1000`; the input unit for COCOMO.
ULOC	Unique Lines Of Code — distinct code lines after removing exact duplicates.
DRYness	"Don't Repeat Yourself"-ness — the percentage of code lines that are unique (ULOC / code lines).
COCOMO	COnstructive COst MOdel — Boehm's algorithmic effort/cost/schedule estimation model; oxide-sloc uses Basic COCOMO I, organic mode.
PM	Person-Months — the unit of COCOMO effort.
TDEV	Time to DEVelop — COCOMO's nominal schedule, in calendar months.
CC	Cyclomatic Complexity — branching complexity; here a lexical approximation (keyword count), not a true McCabe CFG computation.
McCabe / CFG	McCabe's cyclomatic complexity over a Control-Flow Graph: `M = E − N + 2P`.
IEEE 1045-1992	IEEE Standard for Software Productivity Metrics — defines consistent SLOC counting; oxide-sloc implements its physical-SLOC model.
CLI	Command-Line Interface — the `oxide-sloc` terminal tool.
UI	User Interface — here the browser-based web UI.
API	Application Programming Interface — the REST endpoints (§9).
REST	REpresentational State Transfer — the HTTP API style.
MCP	Model Context Protocol — the standard letting AI agents call oxide-sloc as a tool.
CI/CD	Continuous Integration / Continuous Delivery (or Deployment) — automated build-and-ship pipelines.

TLS	Transport Layer Security — encryption for the server (HTTPS).
PEM	Privacy-Enhanced Mail — the text encoding used for TLS certificate and key files.
SMTP	Simple Mail Transfer Protocol — used by `send` for email delivery.
LCOV	A coverage report format (line coverage) oxide-sloc can import.
JaCoCo / Cobertura / NYC	Other coverage report formats oxide-sloc can import (§7).
LAN	Local Area Network — the scope of server mode.
CSP	Content-Security-Policy — an HTTP header that can affect how the HTML report renders inside Jenkins.
VT	VirusTotal — the malware-scanning service used on release binaries.
GHCR	GitHub Container Registry — where the Docker image is published.
AGPL	Affero General Public License — oxide-sloc's licence (AGPL-3.0-or-later).
Hotspot	A file ranked high by code lines × commit churn — a refactoring priority (§7).
Physical SLOC	Counting lines as they appear in the file (after policy resolution), as opposed to logical SLOC which counts statements regardless of formatting.
Vendoring	Committing dependency sources into the repo (`vendor.tar.xz`) so builds need no network.
Air-gapped	A machine with no internet access; oxide-sloc installs and runs fully offline.

APPENDIX A

Supported languages (60)

Language detection, lexical analysis, symbol counting, and test-function detection are available for all 60 of the following.

Ada, Assembly, Awk, C, C++, C#, Clojure, CMake, Crystal, CSS, D, Dart, Dockerfile, Elixir, Elm, Erlang, F#, Fortran, GLSL/HLSL, Go, GraphQL, Groovy, Haskell, HCL/Terraform, HTML, Java, JavaScript, Julia, Kotlin, Lisp/Scheme, Lua, Makefile, Nim, Nix, Objective-C, OCaml, Pascal/Delphi, Perl, PHP, PowerShell, Protocol Buffers, Python, R, Ruby, Rust, Scala, SCSS/Sass, Shell, Solidity, SQL, Svelte, Swift, Tcl, TypeScript, Verilog/SystemVerilog, VHDL, Visual Basic, Vue, XML/SVG, Zig.

INTENTIONALLY EXCLUDED

TOML, Markdown, and YAML are deliberately not counted — no meaningful SLOC metric applies to them — so they never contribute to code totals.

APPENDIX B

Environment variable reference

Every environment variable oxide-sloc reads, across all modes.

Variable	Used by	Purpose
`RUST_LOG`	All	Log level: error/warn/info/debug/trace
`OXIDE_SLOC_ROOT`	All	Working dir for outputs & relative paths
`SLOC_API_KEY`	Web UI	Bearer-token auth for every request
`SLOC_ALLOWED_ROOTS`	Web UI	Allow-list of analyzable host directories
`SLOC_TLS_CERT` / `SLOC_TLS_KEY`	Web UI	PEM cert / key for native TLS
`SLOC_BROWSER`	Web UI / PDF	Path to a Chromium-based browser
`SLOC_BROWSER_NOSANDBOX`	Web UI / PDF	`1` adds `--no-sandbox` (Docker)
`SLOC_REGISTRY_PATH`	Web UI	Override `registry.json` location
`SLOC_SMTP_HOST/USER/PASS`	`send`	SMTP credentials (prefer over CLI flags)
`SLOC_WEBHOOK_TOKEN`	`send`	Bearer token for webhook delivery
`VT_API_KEY`	`release.yml`	VirusTotal API v3 key
`ARTIFACT_REPO_*`	Artifact push	Backend, URL, path, extra config, dry-run, manifest
`SLOC_ARTIFACT_REPO_USER/PASS`	Artifact push	Repository credentials
`OXIDE_SLOC_ENV_FILE`	CI bootstrap	Path to a persisted credentials env file

APPENDIX C

Repository & output layout

Where things live, for orientation when maintaining or contributing.

```
crates/
  sloc-cli/         # CLI entry point and commands (binary: oxide-sloc)
  sloc-config/      # Config schema and TOML parsing
  sloc-core/        # Discovery, decoding, aggregation, delta, COCOMO/hotspots
  sloc-git/         # Git CLI wrappers, webhook parsing, scan-schedule store
  sloc-languages/   # Language detection, lexical analyzers, symbol counting
  sloc-report/      # HTML rendering, PDF/CSV/Excel export
  sloc-web/         # Axum web server, metrics API, badge endpoint
  sloc-mcp/         # MCP stdio server for AI agent integration
ci/                 # CI scripts + config presets
docs/               # airgap.md, ci-integrations.md, server-deployment.md, openapi.yaml
dist/               # Windows pre-built binary (committed by CI each release)
examples/           # Runnable examples + sloc.example.toml
scripts/            # run.sh, serve-server.sh (user-facing entry points)
toolchain/          # Bundled Rust toolchain archives for offline Linux builds
vendor.tar.xz       # All Rust crate sources, for fully offline builds
```

 This guide documents v1.5.73; run `oxide-sloc <command> --help` and see the repository's `docs/` directory for the version-accurate, authoritative reference.